МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ

по лабораторной работе №11
по дисциплине «Организация процессов и программирование в среде Linux»
Тема: Взаимодействие процессов через сокеты
Задание 3

Студент гр. 3311 _____ Баймухамедов Р. Р.

Преподаватель _____ Лазарев Э. А.

Санкт-Петербург
2026

ЦЕЛЬ РАБОТЫ

Изучить взаимодействие процессов через потоковые сокеты TCP, организацию параллельного обслуживания клиентов с помощью fork и ограничение времени ожидания соединения и обмена данными.

ЗАДАНИЕ

Разработать две программы - сервер и клиент, обменивающиеся сообщениями через потоковые сокеты. После соединения клиент генерирует случайную последовательность чисел, выводит её на экран и передаёт серверу. Сервер создаёт для каждого клиента отдельный обслуживающий процесс, который принимает последовательность, сортирует её, выводит на экран и отправляет обратно. Клиент выводит полученный результат и завершает работу.

Предусмотреть завершение клиента при невозможности соединения или истечении времени ожидания отправки сообщения. Сервер должен завершать ожидание новых клиентов, если в течение заданного времени запросы не поступают. После соединения ожидание сообщений с обеих сторон также должно быть ограничено.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Сокет представляет собой конечную точку обмена данными между процессами. В работе используются семейство адресов AF_INET и тип SOCK_STREAM, соответствующие TCP для IPv4. Сервер связывает сокет с адресом и портом функцией bind, переводит его в режим ожидания соединений функцией listen и принимает клиента функцией accept. Клиент устанавливает соединение с помощью connect.

TCP передаёт упорядоченный поток байтов без сохранения границ сообщений. Поэтому один вызов recv может вернуть только часть массива, а send - передать только часть буфера. Функция transfer повторяет

операции до передачи всех данных или возникновения ошибки. В принятом протоколе каждое сообщение содержит ровно 10 чисел uint32_t, то есть 40 байт. Числа переводятся в сетевой порядок байтов функцией htonl и восстанавливаются функцией ntohl.

После accept сервер вызывает fork. Дочерний процесс закрывает слушающий сокет, обслуживает одно соединение и завершается. Родитель закрывает свою копию присоединённого сокета и продолжает принимать клиентов. Обработка SIGCHLD с SIG_IGN и SA_NOCLDWAIT позволяет автоматически освобождать сведения о завершившихся дочерних процессах.

Функция poll ограничивает ожидание готовности сокета. Для соединения используется неблокирующий connect, после которого проверяется SO_ERROR. При обмене данные передаются с флагом MSG_DONTWAIT. Время отсчитывается по CLOCK_MONOTONIC, поэтому перевод системных часов не изменяет тайм-аут. Для каждой операции приёма или отправки задаётся общий срок, который не начинается заново после получения очередной части сообщения.

2 ПРОГРАММЫ

2.1 Подготовка файлов

Исходный код размещается в файлах server.c и client.c. Общие константы, ожидание готовности сокета, передача данных и вывод чисел вынесены в common.h. Заголовочный файл подключается к обеим программам; отдельный исполняемый файл для него не создаётся. Используется порт 3434. Время ожидания в секундах передаётся при запуске каждой программы и может принимать значения от 1 до 3600.

2.2 Общий заголовочный файл common.h

Функция wait_ready ожидает готовность сокета до указанного срока, transfer передаёт или принимает весь массив, а print_numbers выводит числа и PID процесса. Вспомогательная функция timeout_arg проверяет аргумент времени ожидания.

```
#ifndef COMMON_H
#define COMMON_H
#define _POSIX_C_SOURCE 200809L
#include <arpa/inet.h>
#include <errno.h>
#include <poll.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <time.h>
#include <unistd.h>

#define COUNT 10
#define PORT 3434

static int timeout_arg(const char *text)
{
    char *end;
    long value = strtol(text, &end, 10);
    if (*text == '\0' || *end || value < 1 || value > 3600) {
        fprintf(stderr, "Timeout must be 1..3600 seconds\n");
        exit(EXIT_FAILURE);
    }
    return (int)value;
}

static int64_t now_ms(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) < 0) {
        perror("clock_gettime");
        exit(EXIT_FAILURE);
    }
    return (int64_t)ts.tv_sec * 1000 + ts.tv_nsec / 1000000;
}

static int wait_ready(int fd, short event, int64_t deadline)
{
    struct pollfd p = {.fd = fd, .events = event};
    for (;;) {
        int64_t left = deadline - now_ms();
        if (left <= 0) { errno = ETIMEDOUT; return -1; }
        int result = poll(&p, 1, (int)left);
        if (result > 0) {
            if (p.revents & POLLNVAL) {
                errno = EBADF;
                return -1;
            }
```

```
            return 0;
        }
        if (result == 0) { errno = ETIMEDOUT; return -1; }
        if (errno != EINTR) return -1;
    }
}

static int transfer(int fd, void *data, size_t size,
                    int sending, int seconds)
{
    char *p = data;
    int64_t deadline = now_ms() + seconds * 1000;
    while (size > 0) {
        if (wait_ready(fd, sending ? POLLOUT : POLLIN,
                       deadline) < 0) return -1;
        ssize_t n = sending
            ? send(fd, p, size, MSG_DONTWAIT | MSG_NOSIGNAL)
            : recv(fd, p, size, MSG_DONTWAIT);
        if (n > 0) { p += n; size -= (size_t)n; }
        else if (n == 0) { errno = ECONNRESET; return -1; }
        else if (errno != EINTR && errno != EAGAIN &&
                 errno != EWOULDBLOCK) return -1;
    }
    return 0;
}

static void print_numbers(const char *label, uint32_t *a)
{
    char line[256];
    int used = snprintf(line, sizeof(line), "PID %ld %s:",
                        (long)getpid(), label);
    for (int i = 0; i < COUNT; ++i)
        used += snprintf(line + used, sizeof(line) - (size_t)used,
                         " %u", (unsigned)a[i]);
    puts(line);
}
#endif
```

2.3 Текст программы сервера server.c

Родительский процесс ожидает новые подключения. После каждого принятого соединения отсчёт ожидания следующего клиента начинается заново. Обслуживающий процесс принимает массив, сортирует его по возрастанию функцией qsort и возвращает результат клиенту.

```
#include "common.h"

static int compare(const void *left, const void *right)
{
    uint32_t a = *(const uint32_t *)left;
    uint32_t b = *(const uint32_t *)right;
    return (a > b) - (a < b);
}

static int serve(int fd, int seconds)
{
    uint32_t numbers[COUNT];
```

```
    if (transfer(fd, numbers, sizeof(numbers), 0, seconds) < 0) {
        perror("receive from client");
        return EXIT_FAILURE;
    }
    for (int i = 0; i < COUNT; ++i)
        numbers[i] = ntohl(numbers[i]);
    qsort(numbers, COUNT, sizeof(numbers[0]), compare);
    print_numbers("sorted", numbers);
    for (int i = 0; i < COUNT; ++i)
        numbers[i] = htonl(numbers[i]);
    if (transfer(fd, numbers, sizeof(numbers), 1, seconds) < 0) {
        perror("send to client");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}

int main(int argc, char **argv)
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s timeout_seconds\n", argv[0]);
        return EXIT_FAILURE;
    }
    int seconds = timeout_arg(argv[1]);
    setvbuf(stdout, NULL, _IONBF, 0);
    struct sigaction sa = {0};
    sa.sa_handler = SIG_IGN;
    sa.sa_flags = SA_NOCLDWAIT;
    sigemptyset(&sa.sa_mask);
    if (sigaction(SIGCHLD, &sa, NULL) < 0) {
        perror("sigaction");
        return EXIT_FAILURE;
    }
    int listener = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
    if (listener < 0) { perror("socket"); return EXIT_FAILURE; }
    int one = 1;
    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR,
                   &one, sizeof(one)) < 0) {
        perror("setsockopt"); close(listener); return EXIT_FAILURE;
    }
    struct sockaddr_in address = {0};
    address.sin_family = AF_INET;
    address.sin_port = htons(PORT);
    address.sin_addr.s_addr = htonl(INADDR_ANY);
    if (bind(listener, (struct sockaddr *)&address,
             sizeof(address)) < 0 || listen(listener, 16) < 0) {
        perror("bind/listen"); close(listener); return EXIT_FAILURE;
    }
    printf("Server PID %ld, port %d, timeout %d s\n",
           (long)getpid(), PORT, seconds);
    int status = EXIT_SUCCESS;
    int64_t deadline = now_ms() + seconds * 1000;
    for (;;) {
        if (wait_ready(listener, POLLIN, deadline) < 0) {
            if (errno == ETIMEDOUT)
                printf("No new clients for %d s. Server stopped.\n",
                       seconds);
            else { perror("wait client"); status = EXIT_FAILURE; }
            break;
        }
        int fd = accept(listener, NULL, NULL);
```

```
        if (fd < 0) {
            if (errno == EINTR || errno == EAGAIN ||
                errno == EWOULDBLOCK) continue;
            perror("accept"); status = EXIT_FAILURE; break;
        }
        deadline = now_ms() + seconds * 1000;
        pid_t pid = fork();
        if (pid == 0) {
            close(listener);
            int result = serve(fd, seconds);
            close(fd);
            _exit(result);
        }
        if (pid < 0) perror("fork");
        else printf("Client served by PID %ld\n", (long)pid);
        close(fd);
    }
    close(listener);
    return status;
}
```

2.4 Текст программы клиента client.c

Клиент проверяет установление соединения, формирует 10 псевдослучайных целых чисел от 0 до 99, отправляет массив и ожидает ответ. Полученные числа выводятся после преобразования из сетевого порядка байтов. При ошибке или истечении времени ожидания сокет закрывается, программа завершается с ненулевым кодом.

```
#include "common.h"

int main(int argc, char **argv)
{
    if (argc != 3) {
        fprintf(stderr, "Usage: %s server_ip timeout_seconds\n",
                argv[0]);
        return EXIT_FAILURE;
    }
    int seconds = timeout_arg(argv[2]);
    struct sockaddr_in address = {0};
    address.sin_family = AF_INET;
    address.sin_port = htons(PORT);
    if (inet_pton(AF_INET, argv[1], &address.sin_addr) != 1) {
        fprintf(stderr, "Invalid IPv4 address\n");
        return EXIT_FAILURE;
    }
    int fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
    if (fd < 0) { perror("socket"); return EXIT_FAILURE; }
    if (connect(fd, (struct sockaddr *)&address,
                sizeof(address)) < 0) {
        if (errno != EINPROGRESS) goto error;
        if (wait_ready(fd, POLLOUT,
                       now_ms() + seconds * 1000) < 0) goto error;
        int socket_error = 0;
        socklen_t length = sizeof(socket_error);
        if (getsockopt(fd, SOL_SOCKET, SO_ERROR,
```

```
                        &socket_error, &length) < 0) goto error;
            if (socket_error) { errno = socket_error; goto error; }
        }
        puts("Connected to server");
        uint32_t numbers[COUNT];
        srand((unsigned)time(NULL) ^ (unsigned)getpid());
        for (int i = 0; i < COUNT; ++i)
            numbers[i] = (uint32_t)(rand() % 100);
        print_numbers("generated", numbers);
        for (int i = 0; i < COUNT; ++i)
            numbers[i] = htonl(numbers[i]);
        if (transfer(fd, numbers, sizeof(numbers), 1, seconds) < 0)
            goto error;
        if (transfer(fd, numbers, sizeof(numbers), 0, seconds) < 0)
            goto error;
        for (int i = 0; i < COUNT; ++i)
            numbers[i] = ntohl(numbers[i]);
        print_numbers("received", numbers);
        close(fd);
        return EXIT_SUCCESS;
error:
        perror("client: connect/send/receive");
        close(fd);
        return EXIT_FAILURE;
}
```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММ

3.1 Компиляция

Обе программы компилируются компилятором GCC. Заголовочный файл common.h подключается автоматически из текущего каталога.

```
user@linuxlabs:~/lab11$ ls
client.c  common.h  server.c
user@linuxlabs:~/lab11$ gcc server.c -o server
user@linuxlabs:~/lab11$ gcc client.c -o client
user@linuxlabs:~/lab11$ ls
client  client.c  common.h  server  server.c
user@linuxlabs:~/lab11$
```

Рисунок 1 - Компиляция программ сервера и клиента

3.2 Обмен данными с одним клиентом

Для проверки на одном компьютере клиент обращается к адресу обратной петли 127.0.0.1. На экране клиента отображаются исходная и полученная последовательности, на экране сервера - PID обслуживающего процесса и отсортированный массив.

```
user@linuxlabs:~/lab11$ ls
client  client.c  common.h  server  server.c
user@linuxlabs:~/lab11$ ./server 30
Server PID 1694, port 3434, timeout 30 s
Client served by PID 1720
PID 1720 sorted: 8 10 23 35 41 42 60 62 95 97
No new clients for 30 s. Server stopped.
user@linuxlabs:~/lab11$ _
```

Рисунок 2 - Приём и сортировка массива обслуживающим процессом сервера

```
user@linuxlabs:~/lab11$ ./client 127.0.0.1 5
Connected to server
PID 1719 generated: 97 8 41 35 23 42 95 62 60 10
PID 1719 received: 8 10 23 35 41 42 60 62 95 97
user@linuxlabs:~/lab11$ _
```

Рисунок 3 - Отправка исходного массива и получение отсортированного результата клиентом

3.3 Обслуживание нескольких клиентов

Выполнены три запуска клиента. Каждое соединение обслуживается отдельным дочерним процессом сервера с собственным PID. Полученные клиентами массивы совпадают с соответствующими отсортированными массивами на сервере. Эти запуски подтверждают обслуживание нескольких подключений; одновременная работа с молчащим и обычным клиентами показана в пункте 3.6.

```
user@linuxlabs:~/lab11$ ./server 600
Server PID 1833, port 3434, timeout 600 s
Client served by PID 1861
PID 1861 sorted: 14 29 42 47 52 62 65 82 84 87
Client served by PID 1872
PID 1872 sorted: 6 11 28 35 40 48 61 67 87 89
Client served by PID 1883
PID 1883 sorted: 14 29 42 47 52 62 65 82 84 87
```

Рисунок 4 - Обработка трёх подключений разными процессами сервера

```
user@linuxlabs:~/lab11$ ./client 127.0.0.1 5 > client1.log 2>&1 &
[1] 1856
[1]+  Done                    ./client 127.0.0.1 5 > client1.log 2>&1
user@linuxlabs:~/lab11$ ./client 127.0.0.1 5 > client2.log 2>&1 &
[1] 1867
[1]+  Done                    ./client 127.0.0.1 5 > client2.log 2>&1
user@linuxlabs:~/lab11$ ./client 127.0.0.1 5 > client3.log 2>&1 &
[1] 1878
[1]+  Done                    ./client 127.0.0.1 5 > client3.log 2>&1
user@linuxlabs:~/lab11$ cat client1.log client2.log client3.log
Connected to server
PID 1856 generated: 47 82 29 87 42 62 65 14 84 52
PID 1856 received: 14 29 42 47 52 62 65 82 84 87
Connected to server
PID 1867 generated: 67 6 11 28 48 35 89 61 40 87
PID 1867 received: 6 11 28 35 40 48 61 67 87 89
Connected to server
PID 1878 generated: 47 82 29 87 42 62 65 14 84 52
PID 1878 received: 14 29 42 47 52 62 65 82 84 87
user@linuxlabs:~/lab11$
```

Рисунок 5 - Результаты трёх запусков клиента

3.4 Завершение сервера при отсутствии клиентов

Если новые подключения не поступают в течение заданного интервала, родительский процесс выводит продолжительность ожидания и завершает работу. Отсчёт ведётся от запуска сервера или последнего

принятого соединения. Уже созданные обслуживающие процессы завершают свои операции самостоятельно.

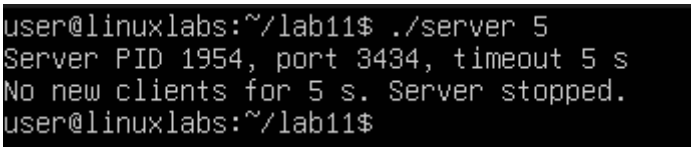


Рисунок 6 - Завершение сервера после 5 секунд без новых подключений

3.5 Подключение при отсутствии сервера

Если сервер не запущен, подключение к локальному порту отклоняется. Клиент выводит сообщение об ошибке и завершает работу, не переходя к генерации массива. Параметр тайм-аута задаёт верхнюю границу ожидания: при немедленном отказе ждать весь интервал не требуется.

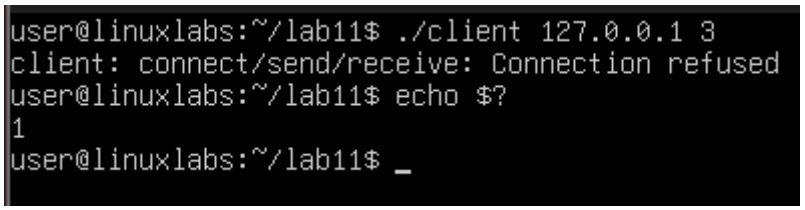


Рисунок 7 - Отказ в подключении при отсутствии сервера и код завершения клиента 1

3.6 Тайм-аут приёма данных на сервере

Для проверки использована тестовая копия клиента silent_client.c с вызовом sleep(10) после установления соединения, перед отправкой массива. Сервер запущен с тайм-аутом 5 секунд. Пока тестовый клиент не передаёт данные, другой обслуживающий процесс успешно обрабатывает запрос обычного клиента. После истечения тайм-аута сервер выводит receive from client: Connection timed out и закрывает соединение с тестовым клиентом. После паузы тестовый клиент получает ошибку Connection reset by peer.

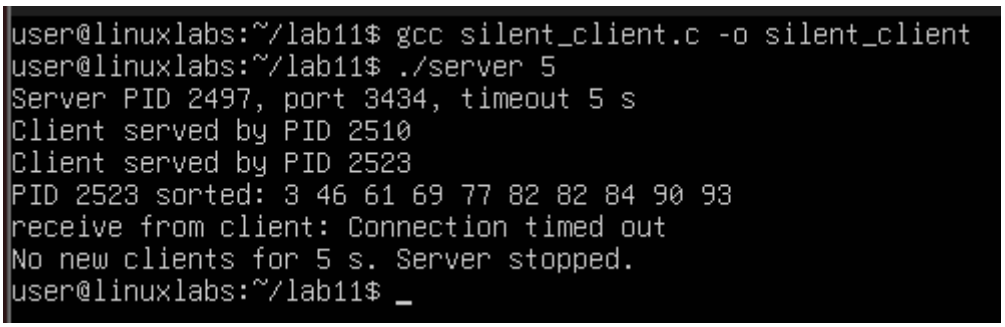


Рисунок 8 - Тайм-аут приёма данных от молчащего клиента и обслуживание обычного клиента

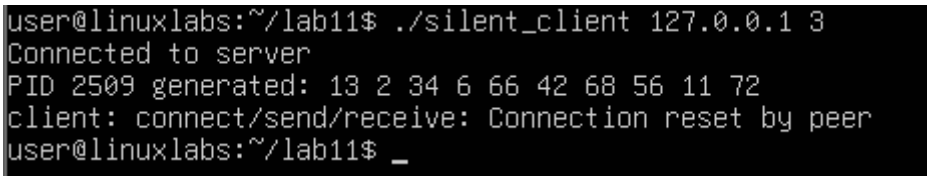


Рисунок 9 - Завершение тестового клиента после закрытия соединения сервером

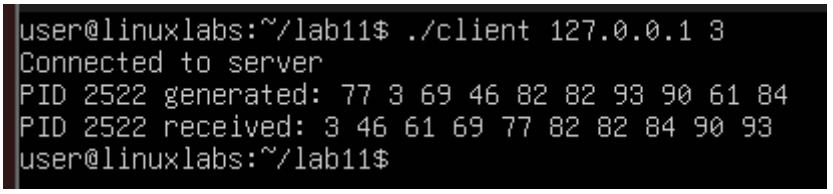


Рисунок 10 - Успешный обмен обычного клиента во время ожидания данных от тестового клиента

3.7 Тайм-аут ожидания ответа клиентом

Для проверки использована тестовая копия сервера silent_server с задержкой sleep(8) перед приёмом данных в обслуживающем процессе. Клиент запущен с тайм-аутом 3 секунды. Соединение устанавливается, клиент формирует и отправляет массив, но ответ не поступает за отведённое время. Клиент выводит Connection timed out и завершается с кодом 1. После задержки сервер принимает и сортирует массив, однако клиент к этому моменту уже завершил ожидание и закрыл соединение.

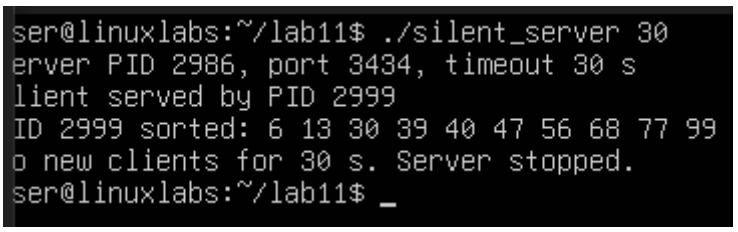


Рисунок 11 - Работа тестового сервера с задержкой обработки запроса

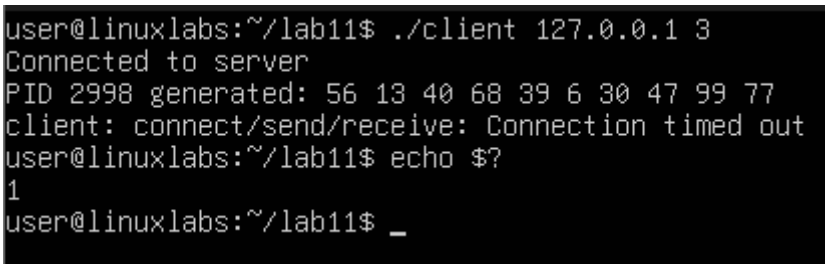


Рисунок 12 - Завершение клиента по тайм-ауту ожидания ответа с кодом 1

4 АНАЛИЗ РЕЗУЛЬТАТОВ

При успешном обмене клиент получает тот же набор чисел, который был отправлен, но упорядоченный по возрастанию. Повторяющиеся значения сохраняются. Совпадение массива received клиента с массивом sorted обслуживающего процесса позволяет проверить правильность передачи и сортировки.

Каждому принятому соединению соответствует отдельный дочерний процесс. Родительский процесс не выполняет сортировку и не ожидает завершения обмена с конкретным клиентом. Поэтому клиент, который не передаёт данные, не блокирует обслуживание остальных подключений.

Ожидание новых соединений, установление соединения, передача массива и получение ответа ограничены по времени. При отсутствии новых клиентов родительский процесс завершает работу штатно, а ошибка обмена завершает только соответствующий клиент или обслуживающий процесс. Тайм-аут относится ко всей текущей операции передачи 40 байт; получение отдельной части массива не увеличивает установленный срок.

ВЫВОД

В ходе лабораторной работы изучено взаимодействие процессов через потоковые сокеты TCP. Разработаны программы клиента и сервера: клиент формирует случайную последовательность чисел, а отдельный процесс сервера сортирует её и возвращает результат. Реализованы параллельное обслуживание подключений, передача полного массива и завершение операций при истечении времени ожидания.